

HESTIA

Arquitetura inicial do MVP HestIA

Decisões estruturais, ASRs, módulos, integrações, dados, segurança e implantação

Tarefa: SCRUM-106

Disciplina: Modelagem e Construção de Software

Épico Jira: SCRUM-99 - USD02

Sprint: Sprint 1

Responsável no Jira: Ricardo Gul D'Avila

Data-base: 2 de setembro de 2026

Status do documento: Versão de trabalho para revisão da equipe

Base de elaboração: materiais do Canvas, estudo de caso do PI-IV, cards do Jira, ADR-001 e estado observado do repositório HestIA. Os rótulos de disciplina do Jira foram tratados como metadados, não como fonte acadêmica principal.

1. Decisão arquitetural

Decisão: adotar um monólito modular em Java 21 e Spring Boot, com camadas internas por módulo, PostgreSQL compartilhado e um Alert Service Node.js integrado por REST síncrono. A decisão principal permanece coerente com a ADR-001.

A arquitetura foi escolhida por adequação ao contexto: equipe pequena, prazo acadêmico, domínio com áreas separáveis, necessidade de um único núcleo transacional e ausência de requisito comprovado de escala ou deploy independente por domínio.

2. Contexto do produto

O HestIA é uma plataforma corporativa para governança, monitoramento e uso responsável de Inteligência Artificial. O estudo de caso organiza o problema nos pilares Pessoas, Tecnologia, Governança e Negócio e prevê indicadores de maturidade, produtividade, segurança, FinOps e capacitação.

Ator	Necessidade principal
Colaborador	Entender políticas, registrar uso e acompanhar capacitação e recomendações.
Gestor/Líder	Acompanhar consumo, risco, produtividade e evolução da equipe.
Executivo	Analisar maturidade, ROI, governança e evolução organizacional.
Administrador	Gerenciar empresas, departamentos, usuários, ferramentas e políticas.

3. Requisitos arquiteturalmente significativos

Os ASRs abaixo são propostas verificáveis para o MVP. Devem ser refinados no Documento de Requisitos e aprovados pela equipe antes de se tornarem compromisso definitivo.

ASR	Cenário / medida proposta	Impacto arquitetural
Segurança	Senha nunca persistida nem retornada em texto aberto; acesso escopado por empresa.	Hash, DTOs de resposta, autorização e auditoria.
Manutenibilidade	Mudança em um módulo não acessa repository interno de outro.	Fachadas públicas, dependências unidirecionais e testes arquiteturais.
Confiabilidade	Falha do Alert Service não corrompe o registro principal.	Timeout, erro controlado e estado de alerta pendente.
Desempenho	Operações CRUD do MVP respondem em até 2 s no ambiente de demonstração.	Consultas simples, índices e limites de payload.
Testabilidade	Casos de uso críticos possuem testes automatizados e pipeline.	Separação de responsabilidades e configuração de teste.
Portabilidade	Execução reproduzível em máquina da equipe.	Docker Compose, variáveis de ambiente e health checks.
Evolução	Módulos podem ser extraídos somente quando ASR justificar.	Contratos explícitos e baixa dependência entre domínios.

4. Análise de alternativas e trade-offs

Alternativa	Ganho	Custo / risco	Decisão
Monólito em camadas globais	Simplicidade inicial.	Mistura domínios e aumenta acoplamento com crescimento.	Rejeitada.
Monólito modular	Limites de domínio, teste e evolução com baixo custo operacional.	Exige disciplina e regras de dependência.	Adotada.
Microserviços	Deploy e escala independentes.	Complexidade operacional, dados distribuídos e maior custo de teste.	Adiada até existir ASR.

5. Módulos de negócio

Módulo	Responsabilidade	Estado
empresa	Empresa, configurações corporativas e orçamento.	Parcialmente implementado.
departamento	Estrutura organizacional vinculada à empresa.	Parcialmente implementado.
usuario	Identidade de negócio, perfil, empresa e departamento.	Parcialmente implementado; segurança bloqueante.
politica	Políticas de uso, versão, situação e vínculo empresarial.	Implementado com duplicidade a corrigir.
ferramenta	Catálogo de ferramentas/modelos, risco, aprovação e custo.	Somente enum; implementação pendente.
consumo	Registro de uso, custo, tokens e resultado de avaliação.	Planejado para o MVP.
indicadores	Métricas e visão de maturidade organizacional.	Incremental; primeiro painel no protótipo.
notificacao	Orquestração de alertas e consulta de eventos.	Gateway Spring e Alert Service Node planejados.

6. Regras de modularidade

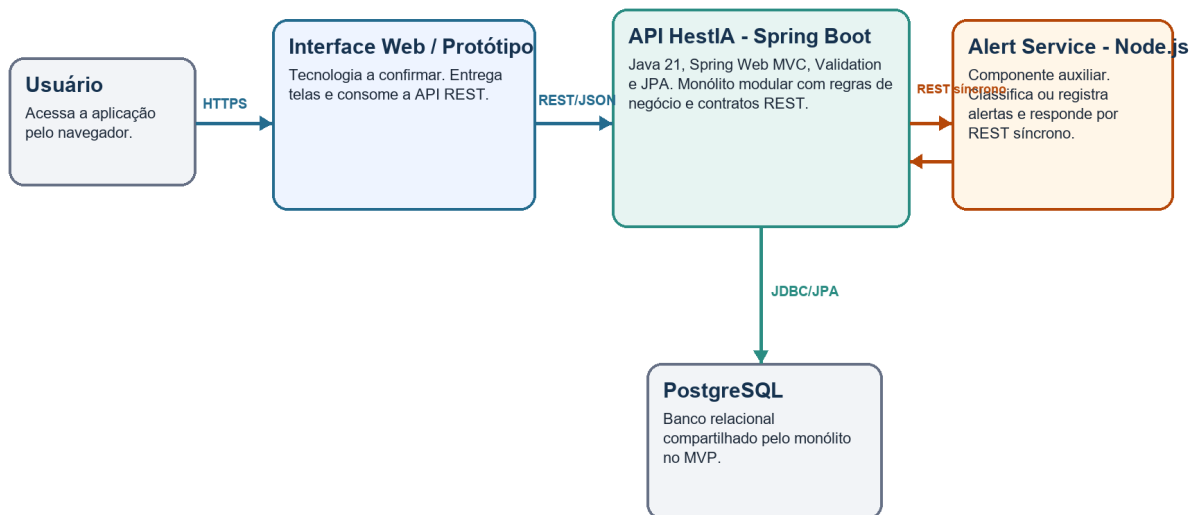
1. Toda funcionalidade pertence a um módulo de negócio explícito.
2. Controller chama caso de uso/fachada; não acessa repository diretamente.
3. Repository pertence ao módulo responsável pela entidade.
4. Um módulo usa outro somente por contrato público; acesso direto ao repository alheio é proibido.
5. Entidade JPA possui um único modelo canônico e não é duplicada em outro módulo.
6. DTOs de entrada e saída protegem o domínio e dados sensíveis.
7. Dependências circulares são proibidas; mudanças de limite exigem ADR.

br.com.hestia +-- empresa | +-- api | application | domain | infrastructure +-- departamento +-- usuario +-- politica +-- ferramenta +-- consumo +-- notificacao +-- shared # somente recursos técnicos neutros

7. Arquitetura de execução

C4 - Nível 2: Contêineres

Arquitetura-alvo do MVP e estado de cada contêiner



Implantação proposta: Docker Compose com serviços hestia-api, postgres e alert-service; interface web adicionada quando o protótipo for versionado. O Alert Service não possui autonomia de domínio suficiente para alterar a decisão principal: o núcleo continua sendo um monólito modular.

Figura 1 - Contêineres propostos para o MVP.

7.1 Integração Spring Boot e Node.js

O material da disciplina demonstra que uma funcionalidade externa em Node.js pode ser integrada ao monólito por REST sem transformar automaticamente o núcleo em microserviços. No Hestia, o Alert Service será um componente auxiliar especializado em avaliar ou registrar alertas.

Aspecto	Decisão do MVP
Protocolo	HTTP REST síncrono com JSON.
Chamada	Spring Boot chama o Alert Service após validar o caso de uso.
Contrato	Entrada com empresa, origem, tipo, valor e contexto; saída com nível e mensagem.
Falha	Timeout curto, erro controlado, log com correlação e possibilidade de alerta pendente.
Saúde	Endpoint /health no Node e health check no Docker Compose.
Evolução	Persistência própria ou mensageria somente se um ASR justificar.

8. Persistência e dados

- PostgreSQL é o banco compartilhado do monólito no MVP.
- Cada tabela possui um módulo proprietário; outros módulos usam contratos públicos.
- Relações devem carregar empresa_id para garantir segregação organizacional.
- Migrações de esquema devem ser versionadas; atualização automática destrutiva não é permitida.

- Política de uso terá uma única entidade canônica no módulo política.
- Logs não devem conter senha, token, conteúdo sensível ou credencial de fornecedor.

9. API, erros e segurança

Tema	Padrão
URLs	Recursos no plural e versão /api/v1 quando os contratos forem estabilizados.
DTOs	Separar entrada, saída e integrações; nunca retornar senha.
Validação	Jakarta Validation no limite da API e regras de negócio na aplicação/domínio.
Erros	Formato consistente com código, mensagem, campos, timestamp e correlationId.
Autenticação	Planejar Spring Security; hash de senha é requisito bloqueante.
Autorização	Perfis FUNCIONARIO, GESTOR e ADMIN, sempre escopados por empresa.

10. Testes, observabilidade e operação

- Testes unitários para regras e casos de uso.
- Testes de integração para JPA, banco e contratos REST.
- Testes arquiteturais para dependências entre módulos.
- Logs estruturados com correlationId e identificação da operação, sem dados sensíveis.
- Health checks de API, banco e Alert Service.
- Pipeline de PR executando build e testes.

11. Mapeamento estado atual -> arquitetura-alvo

Lacuna atual	Correção arquitetural	Prioridade
PoliticaUso duplicada	Um modelo canônico no módulo política.	Bloqueante.
Services acessam repositories de outros módulos	EmpresaFacade e DepartamentoFacade públicas.	Alta.
Senha em texto aberto	Hash e DTO de resposta sem senha.	Bloqueante.
Sem classe principal/configuração consolidada em develop	Reconciliar branches e documentar variáveis.	Bloqueante.
Sem testes/CI suficientes	Teste-base, testes por módulo e pipeline.	Alta.
Node inexistente	Implementar após contrato aprovado na SCRUM-159.	Média após D1.

12. Critérios de aceite da SCRUM-106

1. A arquitetura define estilo, módulos, responsabilidades, dependências e regras de modularidade.
2. Os principais ASRs e trade-offs estão documentados.
3. Spring Boot, PostgreSQL e Node.js possuem papéis, contratos e estratégia de falha definidos.
4. A arquitetura diferencia componentes existentes, planejados e futuros.

5. Os diagramas da SCRUM-105 podem ser derivados sem decisões pendentes.
6. As lacunas do código atual estão registradas com prioridade de correção.

Referências e fontes consultadas

- Canvas ESPM. Deliverables 1 (Entregáveis 1) - Todas as Disciplinas. Curso GTECHSP-TI4A-2592, atividade 293213.
- LELES, Andréia Damasio de. Introdução à Arquitetura de Software - C4_v1. Material da disciplina Modelagem e Construção de Software, 2026.
- LELES, Andréia Damasio de. Monólitos em Camadas x Monólito Modular x Microservices. Material da disciplina, 2026.
- LELES, Andréia Damasio de. Monólitos em camadas e microservices: integração com Node via RestClient. Material da disciplina, 2026.
- ESPM. PI-IV - Projeto Estudo de Caso v2. Projeto Integrado IV, 2026.
- HestIA. ADR-001 - Adotar monólito modular com camadas internas por módulo de negócio. Status: aceita, 30 ago. 2026.
- Jira Projeto_SM. SCRUM-105, SCRUM-106 e SCRUM-150. Consultado em 2 set. 2026.
- Repositório local HestIA. Snapshot técnico de origin/develop e branch local de documentação, consultado em 2 set. 2026.
- BROWN, Simon. The C4 Model for Visualising Software Architecture. c4model.com.